RSCL — Algorithm 17: Per-Aircraft Health Series, Telemetry Recorder and ArduPilot DataFlash Export

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)

Source files described (content hash and state at document build time):

`hu-telepresence-gazebo/spaceops_v8/spaceops_server.py`	sha256 `1bdb818b2cfc`	2026-11-12 00:52 UTC	modified vs 983eede
`hu-telepresence-gazebo/scripts/export_ardupilot_logs.py`	sha256 `23320f607f55`	2026-11-11 06:43 UTC	clean @ 10ee643
`hu-telepresence-gazebo/spaceops_v8/spaceops_panels.py`	sha256 `0d1b27037fc1`	2026-11-11 06:40 UTC	clean @ 10ee643

1. Purpose and context

SpaceOps is the observer/recorder of the demo. It never commands flight (except the operator-initiated, simulator-only fault lab); it samples the supervisor's state, derives a small set of health quantities with explicit provenance, stores them for the session, feeds the chat history answers (Algorithm 12), and can export each aircraft's record in the ArduPilot DataFlash binary format so standard log viewers can open it.

2. Inputs, outputs, data structures

- **Input:** `GET :8765/api/state` (timeout 1.8 s) every `interval` = max(0.2, sample_ms/1000), default 0.25 s.
- **Per-sample row** (`LOG_FIELDS`): utc, epoch_ms, elapsed_s, vehicle, mode, fault, telemetry_fresh, x/y/z, $v_{x,y,z}$ estimates, ground speed estimate, yaw, target x/y/z, battery_sim_pct, closest aircraft and distance, fleet minimum.
- **Health series:** per aircraft, deque of 480 samples {t, fresh, mode, fault, z, speed, track_err, nearest, nearest_peer, battery_sim}; values are None when telemetry is stale.
- **Files** (session directory `logs/spaceops_sessions/SO-<time>/`): `flight_telemetry.jsonl`, `<name>_flight.csv`, `flight_events.jsonl`; export `ardupilot_logs/<Name>.BIN` + README.

3. Algorithm

Algorithm 17.1 POLL (one recorder sample)

Source: spaceops_v8/spaceops_server.py: Recorder.poll, Recorder.loop, Recorder.health_series

```
procedure POLL
  data ← GET /api/state; on failure: if online then event SUPERVISOR_LOST; online ← false; return
  for each aircraft d do
    v ← (p − p_prev)/Δt if 0.01 < Δt < 3.0 else 0;  speed ← ‖v_xy‖
    if fresh(d) and d.mode ∉ {GROUNDED, READY, WAITING} then
      battery_d ← max(0, battery_d − min(1, Δt_poll)·(1.15 + 0.75·speed)/60)   // synthetic, % per s
  for each aircraft a with fresh telemetry: nearest_a ← min over fresh b ≠ a of ‖p_a − p_b‖
  min_sep ← min over fresh pairs;  status ← TELEMETRY LOST if any stale else
           SEPARATION ALERT if min_sep < 0.85 else WATCH if min_sep < 1.15 else CLEAR
  emit LINK_STALE / LINK_FRESH on per-aircraft freshness transitions
  emit GZ_<kind> for supervisor events not seen before (deduplicated by content; first poll only seeds)
  emit SAFETY_<status> on status change
  for each aircraft: track_err ← ‖p − target‖ if fresh and target valid else None
    append health sample (z, speed, nearest = values if fresh else None)       // gaps, not last values
  append rows to JSONL and per-aircraft CSV (under the recorder mutex)
  ingest newly appended HU decision lines (byte offset; no historical import)
```

```
loop: every interval (compensated for processing time, ≥ 0.03 s sleep)
```

Algorithm 17.2 EXPORT (DataFlash .BIN per aircraft)

Source: scripts/export_ardupilot_logs.py: DFWriter, export, latlon, gps_time; spaceops_v8/spaceops_panels.py: snapshot_session, ardupilot_zip

```
procedure EXPORT(session, origin = (34.0583, −117.8213, 213.0 m))
  for each aircraft with a non-empty CSV do
    write FMT records for PARM, MSG, MODE, GPS, POS, ATT, HUCT, BAT, HUSP (types 129..137)
    write 4 MSG lines stating SIMULATED telemetry, vehicle/session, origin, battery/attitude caveats
    write PARM HU_SIMULATED = 1, HU_SAMPLE_HZ = 4, HU_ORIG_LAT/LNG/ALT
    for each row r (TimeUS = (epoch_ms − first epoch_ms)·1000) do
      on mode change: MODE (ArduCopter number map) + MSG "Supervisor mode ..."
      if r not fresh: MSG "TELEMETRY STALE - samples omitted until fresh" once; skip row
      (lat, lon) ← (lat0 + deg(y/R), lon0 + deg(x/(R·cos lat0)))   // ENU flat earth, R = 6378137 m
      course ← atan2(v_x, v_y) mod 360;  heading ← (90 − yaw_ENU) mod 360
      GPS(status 3, GPS week/ms with 18 leap s, lat, lon, alt cm, speed, course, v_z, heading)
      POS, ATT(DesYaw = Yaw = heading, centi-degrees, wrapped negative above 327°), HUCT(target z, z,
track error),
      BAT(RemPct = simulated), HUSP(nearest, fleet minimum)
    for supervisor events naming the aircraft within the time span: MSG "EV kind: message" (≤ 64 chars)
  // HTTP export: session files are copied under the recorder mutex, converted outside it, zipped
```

3.1 Flowchart and state machines

Figure 17.1 is the control flow of one iteration of `Recorder.loop` including `Recorder.poll`. The recorder is stateful; Figure 17.2 gives its lifecycle (the `online` flag and the stop flag), the per-aircraft link flag that produces LINK events, and the fleet safety status that produces SAFETY events. All transitions shown are taken from the code; no transition commands an aircraft.

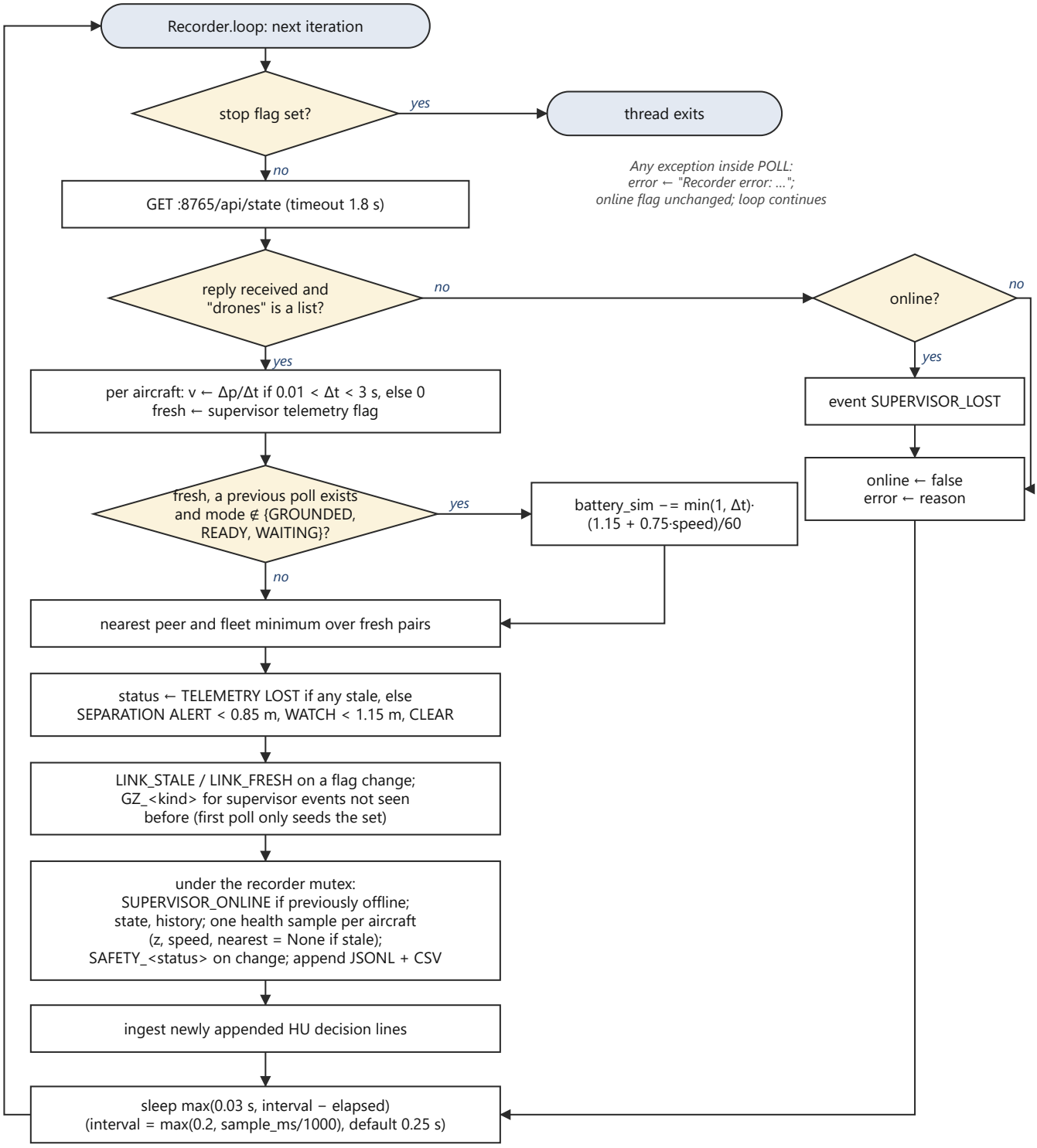


Figure 17.1 Flowchart of one recorder sample (`spaceops_server.py: Recorder.loop, Recorder.poll`). Rounded boxes: start/end; diamonds: decisions. The failure branch records nothing for that tick; the SUPERVISOR_LOST event is emitted only on the online→offline transition.

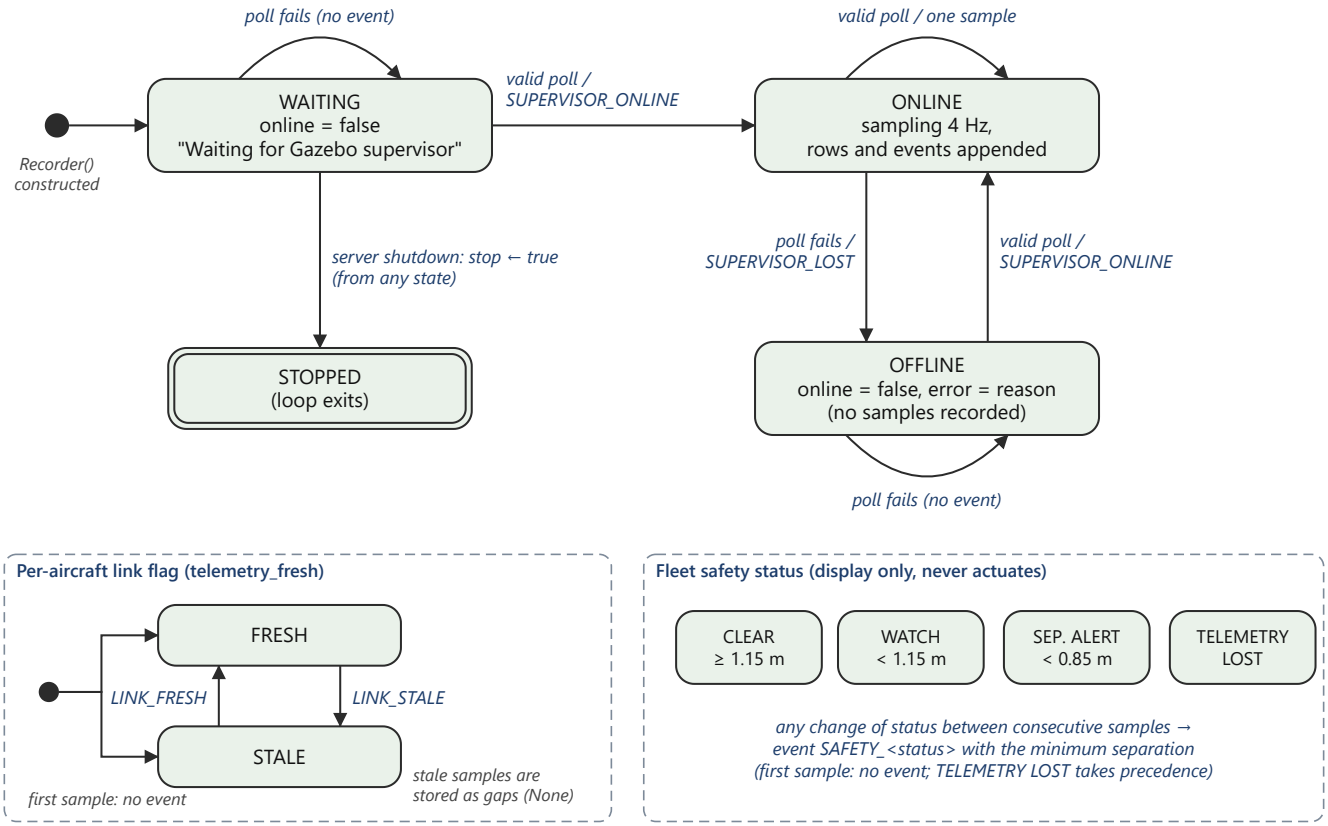


Figure 17.2 State machines. Top: recorder lifecycle (WAITING is the constructor state; the first valid poll emits SUPERVISOR_ONLINE). Bottom left: per-aircraft telemetry flag; the first observation of an aircraft sets the state without an event. Bottom right: the fleet safety status computed each sample; only changes produce events. A Python exception inside POLL sets `error` but leaves the `online` flag, and therefore the lifecycle state, unchanged.

4. Parameters

Parameter	Value
Sampling	0.25 s nominal (4 Hz); health window 480 samples = 120 s
Observer thresholds	alert < 0.85 m, watch < 1.15 m (display only); watchdog reference 0.82 m
Synthetic battery drain	(1.15 + 0.75·ground speed [m/s]) % per minute while not parked
Velocity estimate	finite difference, valid for $0.01 < \Delta t < 3.0$ s
Export origin	34.0583° N, 117.8213° W, 213.0 m (Cal Poly Pomona), flat-earth ENU
Mode map	parked/QUEUED 0, TAKEOFF/MOVE/FORMATION/TURN/SPIN 4, HOVER/FAULT 5, LANDING 9
In-memory export limit	400 MB of input (larger sessions: run the script)

5. Complexity

Per sample $O(N^2)$ for separations and $O(N)$ for rows; memory bounded by the deques (480 × 5 health samples, 240 history, 150 recent rows). Export is linear in the number of rows and events per aircraft (events are scanned once per aircraft: $O(N \cdot E)$).

6. Properties

Guaranteed

- Stale telemetry is recorded as a gap (None) in the health series and is omitted (with an explicit MSG) from the DataFlash export; a last-known value is never presented as current.

- Every derived quantity carries provenance (estimated velocity, simulated battery, sampled separation; `health_series().provenance`, export MSG/PARM lines and README).
- Logs are append-only; a session never imports earlier HU decisions or supervisor events.
- Exports are taken from a consistent snapshot (files copied under the recorder mutex).

Not guaranteed

- Minimum separation is a 4 Hz sample of positions, not a continuous minimum and not a collision-free guarantee.
- Battery figures are a synthetic model, not measured charge. The coordinator (Algorithm 11) uses a different simulated model (2 %/min).
- The .BIN files follow the DataFlash format but are not ArduPilot firmware logs; roll/pitch are absent; flat-earth mapping is only valid for small areas.
- The documented CLI form `--origin LAT,LON,ALT` (with a space) is not parsed; only `--origin=LAT,LON,ALT` is, and the space form would be taken as the session directory.
- A heading of 327–360° is stored as a negative centi-degree value to fit int16.
- In each .BIN file the event MSG records are written after all sample records of that aircraft, so the file order is not chronological (each record still carries its own TimeUS).
- The PARM `HU_SAMPLE_HZ` is written as 4 regardless of `--sample-ms`; the health window is 480 samples, i.e. 120 s only at the default 0.25 s interval.
- During a supervisor outage no samples are recorded; the synthetic battery is drained for at most 1 s of the gap on the first sample after reconnection (Δt is capped at 1 s).
- Velocity is estimated for every aircraft, including stale ones (a frozen position gives 0 m/s); only the health series replaces stale values by gaps.
- Integrity of the logs is by convention (append-only writes); there is no checksum, signature or hash chain that would make later modification detectable.

7. Testing and measured results

- `acceptance/tests/test_spaceops_panels.py`: `ArduPilotExportTests` (`test_zip_has_five_simulated_bins`, `test_no_data_yet`), `EndpointTests` (`test_endpoints_read_only_with_security_headers`, `test_link_transitions_are_recorded_as_events`), fault and latency timeline tests. `acceptance/verify_bin_logs.py` reads exported files back with pymavlink's `DFReader`. No recorded run result of either was found; **not measured here**.
- The fault scenario (`fault_test.py`, run `run_20261111T052914Z`) used the recorder's view: fault visible with detect time 0.0 s; recorder minimum separation 1.442 m at injection.

8. Comparison with alternatives and related work

The recorder is a ground-station observer: it samples the simulator supervisor over HTTP and writes per-session files, then converts them to the ArduPilot DataFlash binary layout so that existing log tools can open them. Established alternatives record on board the vehicle, at the vehicle's own rates, from the flight controller's internal state, or under certification requirements.

Approach	Where / what is recorded	Strength relative to ours	Where ours differs or is weaker
Crash-protected flight recorders, EUROCAE ED-112A [1]	On board; mandatory flight data, cockpit audio, images and data-link messages in a crash-survivable medium	Certified survivability and defined parameter sets; legal evidence	Ours has no survivability, no certification, no mandated parameter list; it is a simulation observer only

ArduPilot DataFlash logs [2]	On board (SD card, flash chip or MAVLink stream); message set selected by `LOG_BITMASK`; tlogs on the ground station	Firmware state (attitude, sensors, controller internals) at flight-controller rates	We reuse the file format only. Our data are 4 Hz samples of the supervisor's state; attitude is yaw only, battery is synthetic, velocity is a finite difference. The PARM/MSG records state this explicitly.
PX4 ULog [3]	On board; self-describing binary format (format messages, uORB topics, microsecond timestamps)	Self-describing schema; per-topic subscription with timestamps	Our CSV/JSONL are self-describing only through column names; the .BIN export uses fixed custom formats (HUCT, HUSP)
Ground-station telemetry logs (MAVLink tlog) [2]	Off board; whatever the telemetry link carried	Closest analogue to our design; standard tooling	Ours is similar in position (off board, link-limited), but adds explicit gap semantics and provenance labels; it has no link-level sequence numbers to quantify losses

Honest assessment. The recorder's specific contributions are small and practical: stale telemetry is recorded as a gap rather than as a held value, every derived quantity is labelled with its provenance, and five aircraft are captured in one consistent session that converts to a standard format. It is weaker than every on-board recorder in temporal resolution (4 Hz sampling of positions, so a minimum separation between samples can be missed), in content (no roll, pitch, sensor or controller data), in authenticity (no tamper evidence) and in fidelity (synthetic battery, flat-earth geodetic mapping). The DataFlash export must not be mistaken for a firmware log; this is why it carries `HU_SIMULATED = 1` and four explanatory MSG lines.

References

1. EUROCAE, *ED-112A: Minimum Operational Performance Specification for Crash Protected Airborne Recorder Systems*, 2013 (superseded by ED-112B, 2023). https://www.eurocae.net/standards/ed-112a/
2. ArduPilot Dev Team, *Downloading and Analyzing Data Logs in Mission Planner*, and the Copter log overview page (DataFlash logs and telemetry logs), ArduPilot documentation. https://ardupilot.org/copter/docs/common-downloading-and-analyzing-data-logs-in-mission-planner.html ; https://ardupilot.org/copter/docs/common-logs.html
3. PX4 Autopilot, *ULog File Format*, PX4 User Guide (main branch). https://docs.px4.io/main/en/dev_log/ulog_file_format.html

9. Relation to other algorithms

Observes the state produced by Algorithms 5–9; supplies history evidence to Algorithm 12 and evidence values to Algorithm 13. The chat storage of the same server also hosts the messages of Algorithms 14 and 15.

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.